

1. ARQUITECTURA USADA — DESCRIPCIÓN COMPLETA

Estilo: Arquitectura Hexagonal (Ports & Adapters) / Clean Architecture aplicada a WinForms.

Capas:

- Presentación (UI): formularios WinForms.
 - Aplicación (Use Cases): clases en ArqHex.*.application.
 - Dominio: entidades y reglas de negocio.
 - Infraestructura: repositorios Oracle.
 - Composition Root: ArqHex.Share.AppServices.
- Flujo: UI → caso de uso → repositorio → Oracle → retorno UI.
- Observaciones:
- Buena separación de responsabilidades.
 - Servicios estáticos dificultan pruebas (recomendado DI).

2. DESCRIPCIÓN DE MÓDULOS, FUNCIONES E INTERFAZ

AppServices:

- Actúa como composition root.
- Crea repositorios Oracle.
- Expone servicios agrupados por área.

Módulos:

• Collectors:

- Repositorio: CollectorRepository.
- Casos de uso: Save, Update, QueryAll, QueryByStatus, QueryByIn.

- **CollectsRepositorio:** CollectRepository.

- Casos de uso: Save, Update, QueryAll, QueryByStatus, QueryByWorkerCode.

• Harvests:

- Repositorio: HarvestRepository.
- Casos de uso: Save, Update, QueryAll, QueryByStatus.

• Plots:

- Repositorio: PlotRepository.
- Casos de uso: QueryAll, QueryById.

• Payments:

- Repositorio: PaymentRepository.
- Casos de uso: Save, Update, QueryAll, QueryByWorkerCode, deleteByPaymentId.

• PaymentDetails:

- Repositorio: PaymentDetailRepository.
- Casos de uso: Save, Update, QueryAll, QueryByPaymentId.

UI WinForms:

- ViewMain: navegación.
- ViewCollector: CRUD recolectores.
- ViewHarvest: campañas.
- ViewOrigin: parcelas.
- ViewPayment: creación y gestión de pagos.

3) Guía de Validaciones del Sistema : Tipos de Validaciones y Dónde Implementarlas

1) Validación Sintáctica (Interfaz de Usuario)

Verifica el formato básico y la estructura de los datos:

- Formato correcto de campos (email, teléfono, fechas)
- Tipos de datos apropiados (números, texto, booleanos)
- Campos obligatorios completados
- Longitud mínima y máxima de cadenas

2) Validación de Negocio (Casos de Uso / Dominio)

Aplica las reglas específicas del dominio:

- Lógica empresarial específica
- Restricciones de valores según contexto
- Cálculos y validaciones complejas
- Reglas que definen el funcionamiento del negocio

3) Validación Referencial (Casos de Uso)

Asegura la consistencia entre entidades relacionadas:

- Verificación de existencia de registros relacionados
- Integridad referencial entre tablas
- Consistencia de datos entre entidades vinculadas

4) Validación Transaccional

Garantiza la atomicidad de operaciones complejas:

- Coherencia entre Payment y PaymentDetails
- Todas las operaciones se completan o ninguna
- Rollback automático en caso de error

5) Validación Previa a Operaciones Críticas

Previene acciones que podrían comprometer la integridad:

- `checkIfPaymentCanBeDeleted`: evita borrar pagos ya procesados
- Verificaciones antes de modificaciones irreversibles
- Validación de estados antes de transiciones

6) Validación de Seguridad (Capa de Aplicación/Servicio)

Controla el acceso y permisos:

- Autorización para operaciones específicas
 - Verificación de permisos por usuario
 - Control de acceso a recursos sensibles
-

4) Reglas de Validación por Entidad

a) Collector (Recolector)

Identificación y Datos Básicos:

- `workerCode`: No puede ser nulo o vacío, debe ser único en el sistema
- `id`: Debe ser mayor que 0
- `firstName` y `lastName`: Deben cumplir con longitud mínima establecida

Estado:

- `status`: Debe estar dentro del conjunto permitido:
 - 0 = Inactivo
 - 1 = Activo
-

b) Collect (Recolección)

Referencias:

- `workerCode`: Debe existir en la tabla de Collectors
- `idPlot`: Debe corresponder a un terreno existente
- `idHarvest`: Si se proporciona, debe estar en estado válido

Datos de Recolección:

- `date`: Debe ser menor o igual a la fecha actual (o seguir regla de negocio específica)
 - `collectedKilos`: Debe ser mayor que 0 cuando `isCountable` = 1
-

c) Harvest (Cosecha)

Datos Económicos:

- pricePerKilo: Debe ser mayor o igual a 0

Fechas:

- startDate: Debe ser anterior a endDate (cuando se use endDate)

Estado:

- Debe tener un estado válido:
 - 0 = Planificado
 - 1 = Activo
 - 2 = Cerrado
-

d) Plot (Terreno)

Identificación:

- Campos identificadores no pueden estar vacíos
 - Coordenadas geográficas deben tener formato válido (si aplica)
 - Validar longitud de campos descriptivos
-

e) Payment (Pago)

Integridad Financiera:

- totalAmount: Debe ser mayor o igual a 0
- El total debe coincidir exactamente con la suma de todos los PaymentDetails

Operaciones:

- No permitir duplicados para el mismo paymentId (excepto en actualizaciones autorizadas)
 - Ejecutar checkIfPaymentCanBeDeleted antes de deleteByPaymentId
-

f) PaymentDetail (Detalle de Pago)

Montos:

- amountToPay: Debe ser mayor que 0

Referencias:

- collectId: Debe referenciar una recolección existente
- harvestId: Debe referenciar una cosecha existente
- workerCode: Debe coincidir con el del pago principal

Consistencia:

- Evitar inconsistencias entre el detalle y el pago principal
-

5. Mejores Prácticas para Mensajes y Experiencia de Usuario

Centralización de Mensajes

- Mantener todos los mensajes de validación en un lugar centralizado
- Usar constantes o archivos de recursos (.resx) para facilitar la traducción
- Permitir fácil mantenimiento y actualización de textos

Mensajes Claros en la Interfaz

Hacer:

- "El campo Kilos debe ser mayor que 0"
- "El código de trabajador no existe en el sistema"
- "La fecha de recolección no puede ser futura"

Evitar:

- "Error en validación"
- "Datos inválidos"
- Mostrar excepciones técnicas o stacktraces

Mapeo de Errores

- Convertir excepciones técnicas en mensajes comprensibles
- Sugerir acciones correctivas cuando sea posible
- Proporcionar contexto sobre qué salió mal y cómo solucionarlo

6) REQUISITOS MÍNIMOS DEL SISTEMA

a) Software:

- Windows 10/11.
- .NET Framework 4.8.
- Oracle Database XE o Server.
- ODP.NET / Oracle ManagedDataAccess.

b) Hardware:

- CPU 2 núcleos.
- RAM 4 GB mínimo (8 GB recomendado).
- 200 MB de espacio libre.
- Conectividad a puerto 1521.

c) Despliegue:

- Windows Server o Windows 10/11.
- Oracle accesible.
- Usuario DB con permisos.
- Scripts SQL preparados.